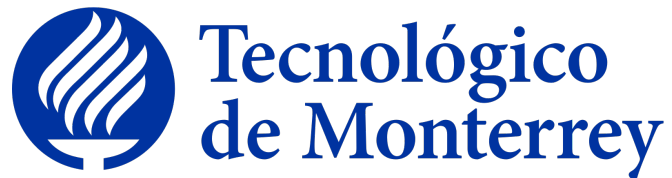


Instituto Tecnológico y de Estudios Superiores de Monterrey



Escuela de Ingeniería y Ciencias

Ingeniería en Ciencia de Datos y Matemáticas

Uso de Álgebras Modernas para Seguridad y Criptografía

Reporte Técnico

Etapa 1 del Reto: Gestor de Identidades y Documentos

Socio Formador: Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P.

Profesores: Raúl Gómez, Anas Wajid y Alberto F. Martínez

Equipo de trabajo: Equipo 5

Grupo: 603

Integrantes

Alberto Rodríguez Reyes (A01383805)

Diego Alberto Rodríguez Ruiz (A01571638)

Rubén Jiménez Sarmiento (A01286256)

Hector Alonso Flores Meléndez (A01571545)

Valeria García Hernández (A01742811)

Lugar y fecha de elaboración: Monterrey, Nuevo León, 14 de junio de 2026

Índice general

Resumen	1
1. Introducción	2
2. Marco de Referencia	3
2.1. Criptografía de Clave Pública y Estándares Internacionales	3
2.2. Bibliotecas y Soluciones de Referencia: Pycryptodome	4
2.3. Casos de Estudio Reales: El Sistema de Identidades del SAT	4
3. Desarrollo	5
3.1. Propuesta de Solución	5
3.1.1. Justificación de decisiones tecnológicas e infraestructura	5
3.2. Desarrollo de Componentes: Backend	7
3.2.1. Autenticación y control de acceso	7
3.2.2. Firma digital RSA con OpenSSL	8
3.3. Desarrollo de Componentes: Frontend	8
3.3.1. Arquitectura de componentes y enrutamiento	9
3.3.2. Módulo Gestor de Documentos	9
3.3.3. Flujo de firma digital en el frontend	9
3.4. Conexión de los Componentes	11
4. Resultados	13
4.1. Resultados del Backend	13
4.2. Resultados del Frontend	13
4.3. Resultados de la Solución Completa	14
4.4. Casos de Uso y Tabla de Validaciones	14
5. Conclusiones y Trabajo Futuro	17
5.1. Conclusiones	17
5.2. Trabajo Futuro	17
Referencias	19

A. Diccionario de Datos — Tabla usuarios	20
B. Fragmento de Código — Verificación Criptográfica de Firma	21

Índice de tablas

2.1. Fortaleza de clave en RSA comparado con algoritmos de clave privada (tomado de Barker et al., 2019).	3
3.1. Páginas principales del frontend React y roles autorizados (elaboración propia).	9
4.1. Matriz de validación y casos de uso del Gestor de Identidades y Documentos (elaboración propia).	15
A.1. Diccionario de datos — tabla usuarios (elaboración propia).	20

Índice de figuras

1.1. Arquitectura general del sistema Gestor de Identidades y Documentos (elaboración propia). .	2
3.1. Pantalla de inicio de sesión del Gestor de Identidades y Documentos (elaboración propia). .	6
3.2. Dashboard del sistema con sesión de administrador iniciada (elaboración propia).	6
3.3. Vista inicial del Gestor de Documentos con estadísticas por etapa (elaboración propia). . . .	9
3.4. Formulario de registro de un nuevo documento en etapa Ventanilla (elaboración propia). . . .	10
3.5. Interfaz de firma digital disponible para el coordinador en etapa Operador (elaboración propia). .	10
3.6. Ticket y folio de firma digital generados tras la operación criptográfica (elaboración propia). .	11
4.1. Verificación de firma digital: el sistema confirma la integridad del documento (elaboración propia).	14
4.2. Documento completado en etapa de Inserción Final tras recorrer el flujo VOCA completo (elaboración propia).	15

Resumen

Este reporte documenta el diseño, implementación y validación del sistema **Gestor de Identidades y Documentos** desarrollado para Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P., como Etapa 1 del reto del bloque MA2006B. La solución es una aplicación web de arquitectura cliente-servidor desacoplada: un frontend de página única (SPA) en React 19 con Vite 8 y un backend compuesto por 22 endpoints PHP sobre Apache, conectados a una base de datos MySQL/MariaDB. El sistema cubre autenticación segura con BCrypt, control de acceso basado en roles (RBAC) con cuatro niveles de privilegio, gestión de usuarios con certificados digitales únicos y control de vigencia temporal, un flujo documental de cuatro etapas (Ventanilla → Operador → Coordinador → Inserción final) con firma digital RSA-2048 y verificación criptográfica de integridad mediante SHA-256. La validación funcional se realizó mediante 15 casos de prueba de caja negra sobre los módulos de identidad, con resultado *Pass* en todos los escenarios documentados. Los fundamentos criptográficos aplicados incluyen las recomendaciones del NIST SP 800-56B para generación de pares de llaves y los principios de entropía de Shannon para la aleatoriedad de certificados, en alineación directa con los contenidos del bloque de Álgebras Modernas para Seguridad y Criptografía.

Capítulo 1

Introducción

Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P., es una organización no gubernamental con sede en Monterrey, Nuevo León, que brinda atención integral a personas migrantes que transitan por el norte de México. La naturaleza de su operación involucra la gestión de información sensible: identidades del personal interno, expedientes de migrantes atendidos, documentos de caso y autorizaciones con distintos niveles de responsabilidad.

Previo al desarrollo de este proyecto, la organización no contaba con un sistema centralizado para la administración de identidades digitales de su personal ni para la firma y trazabilidad de documentos institucionales. Los accesos se gestionaban de manera informal, sin control de vigencias, estados de cuenta ni mecanismos de autenticación robustos, lo que representaba un riesgo operativo y de seguridad para los datos de las personas atendidas.

El reto técnico de este bloque consiste en diseñar e implementar la **Etap 1 del Gestor de Identidades y Documentos**: un sistema que centralice la autenticación, el control de acceso por roles, la generación de certificados digitales únicos por usuario y el control de vigencia de cuentas. El alcance se extendió, a solicitud del socio formador, con un módulo de firma digital de documentos basado en criptografía asimétrica RSA, que cubre la necesidad operativa de validar de forma objetiva la autorización de documentos institucionales del flujo de atención a migrantes.

La Figura 1.1 ilustra la arquitectura de alto nivel del sistema resultante.

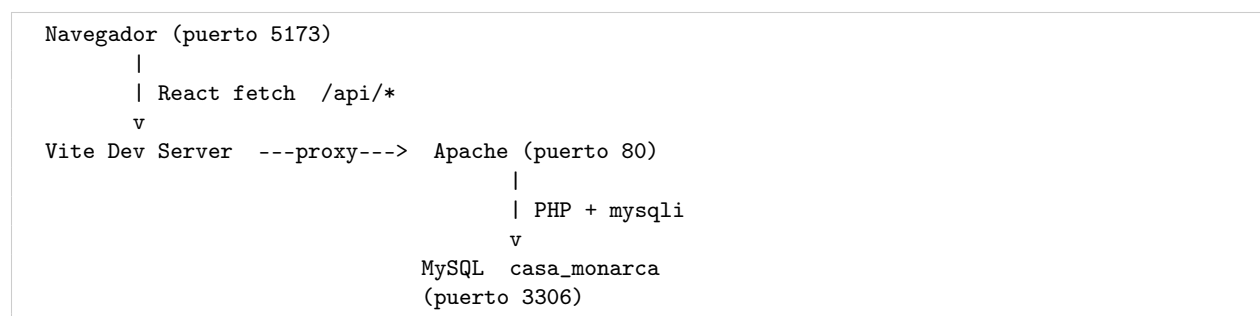


Figura 1.1: Arquitectura general del sistema Gestor de Identidades y Documentos (elaboración propia).

La solución responde a tres necesidades concretas del socio formador: (1) garantizar que solo personal activo y vigente acceda al sistema; (2) diferenciar las acciones permitidas según el rol del usuario; y (3) proveer un mecanismo criptográficamente verificable para la firma de documentos institucionales.

El código fuente completo del proyecto se encuentra disponible en el repositorio público: <https://github.com/Valg234/MA2006B>.

Capítulo 2

Marco de Referencia

2.1. Criptografía de Clave Pública y Estándares Internacionales

La infraestructura de clave pública (PKI) constituye el fundamento teórico de la firma digital implementada en este proyecto. La PKI se basa en la asimetría matemática para garantizar los principios de confidencialidad, integridad y no repudio. Ramió Aguirre (2006) define la criptografía como la disciplina matemática e informática cuyo objeto central es cifrar y proteger mensajes o archivos mediante algoritmos y una o más claves, definición que enmarca con precisión el rol de la PKI dentro del sistema desarrollado.

En los sistemas modernos de gestión de identidad, el algoritmo RSA desempeña un papel central en el establecimiento de llaves y firmas digitales. La robustez de este esquema radica en la dificultad computacional de la factorización de números enteros grandes. El Instituto Nacional de Estándares y Tecnología (NIST) define directrices específicas sobre la longitud mínima de las claves para mitigar ataques de fuerza bruta, estableciendo una equivalencia clara entre los bits de seguridad asimétrica y simétrica (Barker et al., 2019), como se aprecia en la Tabla 2.1.

Tabla 2.1: Fortaleza de clave en RSA comparado con algoritmos de clave privada (tomado de Barker et al., 2019).

Módulo clave pública (RSA, bits)	Tamaño de clave simétrica equivalente (AES, bits)
2048	112
3072	128
4096	152
6144	176
8192	200

En este proyecto se eligió **RSA-2048** como tamaño mínimo de clave, siguiendo el umbral mínimo aceptable establecido por el NIST SP 800-56B Revision 2 para implementaciones que deben mantenerse seguras más allá del año 2030.

Por otra parte, la aleatoriedad de los datos involucrados en la generación de llaves criptográficas y certificados de usuario se rige bajo los principios de la Teoría de la Información. Claude E. Shannon estableció las bases conceptuales para cuantificar la incertidumbre y la predictibilidad de un sistema a través del cálculo de la entropía (Shannon, 2001). En sistemas distribuidos como el desarrollado para este reto, asegurar una alta entropía en la generación de números pseudoaleatorios para llaves privadas y certificados únicos de usuario es crítico para evitar vectores de ataque basados en patrones predecibles.

2.2. Bibliotecas y Soluciones de Referencia: Pycryptodome

En el ecosistema de desarrollo de software actual, la manipulación de primitivas criptográficas avanzadas de clave pública se implementa regularmente mediante bibliotecas modulares de código abierto. Un ejemplo prominente en la literatura técnica es **Pycryptodome**, una biblioteca nativa de Python que empaqueta algoritmos de cifrado simétrico y asimétrico (como RSA y ECC), funciones de derivación de claves y esquemas de firma digital.

Al analizar Pycryptodome, se observa cómo la abstracción de bajo nivel permite a los desarrolladores instanciar módulos de generación de llaves de manera segura sin necesidad de programar las operaciones algebraicas desde cero. De esta solución existente se extrajo la lógica estructural para el manejo de llaves asimétricas y la exportación de certificados en formatos estándar (como PEM), que se mapeó hacia las funciones nativas correspondientes de la extensión **OpenSSL** disponible en PHP.

2.3. Casos de Estudio Reales: El Sistema de Identidades del SAT

A nivel institucional en México, el Servicio de Administración Tributaria (SAT) opera uno de los sistemas distribuidos de gestión de identidades y firma electrónica avanzada (e.firma) más robustos del sector público. El SAT actúa como una Autoridad Certificadora (CA) interna que administra a los contribuyentes vinculando de forma unívoca su identidad legal (RFC, CURP y datos biométricos) a un par de claves criptográficas y a un certificado digital bajo el estándar X.509.

El sistema del SAT demuestra la viabilidad técnica de descentralizar la firma de documentos y la autenticación: el contribuyente resguarda localmente su clave privada (.key) bajo una contraseña simétrica, mientras que el SAT almacena de forma pública el certificado (.cer). Esta arquitectura sirvió de inspiración directa para el alcance del reto con Casa Monarca: la separación estricta de roles, el almacenamiento cifrado de llaves privadas de coordinadores y el control de vigencias temporales son pilares replicables para mitigar la suplantación de identidad en la gestión de expedientes de apoyo humanitario.

Capítulo 3

Desarrollo

3.1. Propuesta de Solución

El equipo propuso una aplicación web de arquitectura cliente-servidor desacoplada compuesta por tres capas independientes: (1) un frontend de página única (SPA) en React con Vite, (2) un backend de endpoints PHP servido por Apache, y (3) una base de datos relacional MySQL/MariaDB. Este desacoplamiento permite que ambas capas evolucionen independientemente y facilita el despliegue en un servidor de hospedaje compartido accesible para el socio formador.

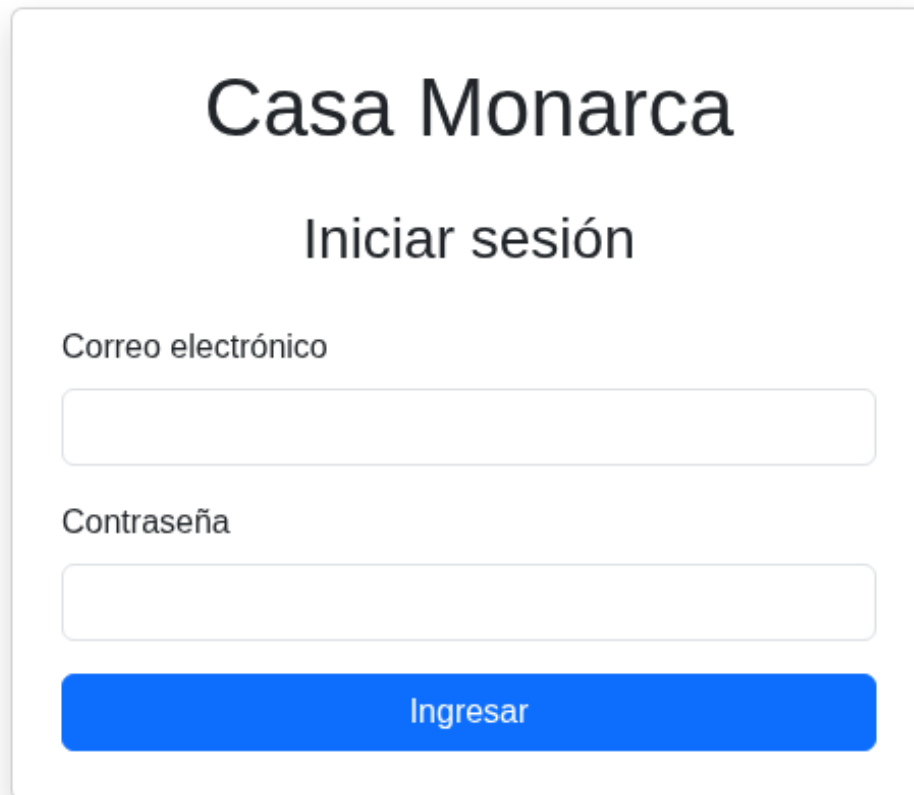
3.1.1. Justificación de decisiones tecnológicas e infraestructura

La decisión central fue elegir entre una arquitectura en nube elástica (AWS) o una solución de servidor tradicional (HostGator / LAMP local). El equipo optó por la segunda opción con base en tres argumentos técnicos, operativos y financieros:

1. **Perfil económico del socio formador.** Casa Monarca opera bajo presupuestos estrictamente asignados a la ayuda humanitaria directa. Las arquitecturas en nube como AWS imponen un esquema de costos variables basados en consumo de cómputo (Pay-as-you-go), lo que introduce riesgo de facturación imprevista para una ONG. Una solución en infraestructura tradicional posee costos fijos predecibles y sumamente bajos.
2. **Complejidad operativa y mantenibilidad.** AWS demanda personal técnico certificado para configurar adecuadamente políticas IAM, VPCs y balanceadores de carga. Al finalizar el bloque, el sistema debe ser mantenido por el personal técnico interno de Casa Monarca. El stack React + PHP + MySQL bajo una arquitectura cliente-servidor estándar reduce drásticamente la complejidad operativa, permitiendo realizar tareas de respaldo y administración mediante herramientas visuales intuitivas como phpMyAdmin.
3. **Privacidad de datos sensibles.** Dado que se gestionan identidades digitales e información confidencial de migrantes, mantener el entorno en un servidor local o de hospedaje dedicado y controlado previene exposiciones accidentales de datos en configuraciones de red erróneas, que son más probables en entornos de nube multi-tenant durante fases tempranas de desarrollo.

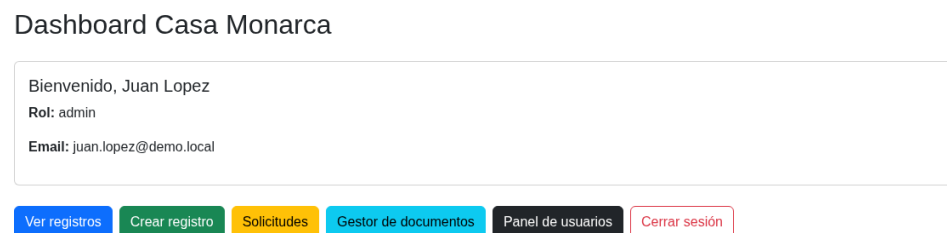
En cuanto al stack de desarrollo, se eligió **React 19 con Vite 8** para el frontend por su ecosistema maduro, la velocidad de iteración que ofrece el Hot Module Replacement (HMR) y la disponibilidad de React Router DOM para la gestión de rutas protegidas sin recargas de página. Para el backend se utilizó **PHP 7.4+** con la extensión OpenSSL, que provee de forma nativa las primitivas de criptografía asimétrica requeridas (RSA, SHA-256, PKCS#1 v1.5) sin dependencias externas adicionales.

La Figura 3.1 y la Figura 3.2 muestran las pantallas iniciales de la solución propuesta e implementada.



The image shows a login interface for 'Casa Monarca'. It features a white card with a light gray border. At the top, the text 'Casa Monarca' is displayed in a large, bold, black font, followed by 'Iniciar sesión' in a slightly smaller, bold, black font. Below this, there are two input fields: the first is labeled 'Correo electrónico' and the second is labeled 'Contraseña'. Both fields are empty and have a light gray border. At the bottom of the card is a prominent blue button with the text 'Ingresar' in white.

Figura 3.1: Pantalla de inicio de sesión del Gestor de Identidades y Documentos (elaboración propia).



The image shows a dashboard for 'Casa Monarca'. At the top, the text 'Dashboard Casa Monarca' is displayed. Below this, there is a white card with a light gray border. Inside the card, the text 'Bienvenido, Juan Lopez' is displayed, followed by 'Rol: admin' and 'Email: juan.lopez@demo.local'. Below the card, there is a horizontal row of six buttons: 'Ver registros' (blue), 'Crear registro' (green), 'Solicitudes' (yellow), 'Gestor de documentos' (cyan), 'Panel de usuarios' (dark gray), and 'Cerrar sesión' (pink).

Figura 3.2: Dashboard del sistema con sesión de administrador iniciada (elaboración propia).

3.2. Desarrollo de Componentes: Backend

El backend está compuesto por 22 endpoints PHP independientes ubicados en la carpeta `api/`. Cada archivo implementa el mismo patrón: lectura de JSON desde `php://input`, validación de datos, consulta a MySQL mediante *prepared statements* con `mysqli` y respuesta JSON estructurada con los campos `success` (booleano) y `mensaje` (string). Se usa `ob_start()` al inicio y `ob_clean()` antes de cada respuesta para garantizar JSON puro, sin salida HTML accidental de PHP.

3.2.1. Autenticación y control de acceso

El endpoint `login.php` implementa el flujo de autenticación en cuatro verificaciones secuenciales, constituyendo el componente central del módulo de identidades:

Listing 3.1: Fragmento de `login.php` — verificación de credenciales, estado y vigencia (elaboración propia).

```
1 $stmt = $conexion->prepare(
2     "SELECT id, nombre, email, rol, estado,
3         fecha_vigencia, certificado_codigo, coordinador_id
4     FROM usuarios WHERE email = ?"
5 );
6 $stmt->bind_param("s", $email);
7 $stmt->execute();
8 $usuario = $stmt->get_result()->fetch_assoc();
9
10 // 1. Verificar existencia del usuario
11 if (!$usuario) {
12     echo json_encode(["success" => false,
13         "mensaje" => "No existe un usuario con ese correo."]);
14     exit;
15 }
16 // 2. Verificar estado activo
17 if ($usuario['estado'] !== 'activo') {
18     echo json_encode(["success" => false,
19         "mensaje" => "Acceso denegado. El usuario no esta activo."]);
20     exit;
21 }
22 // 3. Verificar vigencia temporal
23 if ($usuario['fecha_vigencia'] < date('Y-m-d')) {
24     echo json_encode(["success" => false,
25         "mensaje" => "Acceso denegado. La vigencia ha expirado."]);
26     exit;
27 }
28 // 4. Verificar hash BCRYPT
29 if (!password_verify($password, $usuario['password'])) {
30     echo json_encode(["success" => false,
31         "mensaje" => "Contraseña incorrecta."]);
32     exit;
33 }
34 echo json_encode(["success" => true, "usuario" => $usuario]);
```

Este flujo garantiza que solo usuarios con estado `activo` y vigencia futura puedan autenticarse, validando en orden: existencia, estado, vigencia y contraseña. Cada verificación produce un mensaje diferenciado, facilitando el diagnóstico sin exponer información de seguridad innecesaria.

3.2.2. Firma digital RSA con OpenSSL

El endpoint `firmar_documento.php` implementa la firma digital de documentos en cuatro pasos que aplican directamente los conceptos de clave pública estudiados en el bloque:

Listing 3.2: Fragmento de `firmar_documento.php` — firma RSA-2048 con SHA-256 (elaboración propia).

```
1 // 1. Obtener clave privada cifrada del coordinador desde MySQL
2 $clave_privada_cifrada = $usuario['clave_privada_cifrada'];
3
4 // 2. Descifrar la clave privada con la contraseña del coordinador
5 $clave_privada = openssl_pkey_get_private(
6     $clave_privada_cifrada,
7     $password_firma
8 );
9 if (!$clave_privada) {
10     echo json_encode(["success" => false,
11         "mensaje" => "Contraseña de firma incorrecta."]);
12     exit;
13 }
14
15 // 3. Calcular hash SHA-256 del archivo físico
16 $hash = hash_file('sha256', $ruta_archivo);
17
18 // 4. Firmar el hash con la clave privada RSA-2048
19 openssl_sign($hash, $firma_raw, $clave_privada, OPENSSL_ALGO_SHA256);
20 $firma_base64 = base64_encode($firma_raw);
21
22 // Generar folio único y almacenar en MySQL
23 $folio = "FIR-" . date("Ymd-His")
24     . "-DOC{$documento_id}-USR{$usuario_id}";
```

La verificación posterior (`verificar_firma_documento.php`) recalcula el hash SHA-256 del archivo físico actual y lo valida contra la firma almacenada usando `openssl_verify` con la clave pública del coordinador, detectando cualquier modificación posterior a la firma. La generación de llaves RSA-2048 para los coordinadores se realiza en `generar_llaves_coordinador.php`, almacenando la clave pública en texto PEM y la clave privada cifrada con la contraseña elegida por el coordinador.

3.3. Desarrollo de Componentes: Frontend

El frontend es una aplicación de página única (SPA) construida con React 19 y compilada por Vite 8. La navegación entre páginas se gestiona con React Router DOM 7.14, con protección de rutas mediante el componente `ProtectedRoute.jsx`, que consulta la sesión contra `/api/validar_sesion.php` antes de renderizar cada página protegida.

3.3.1. Arquitectura de componentes y enrutamiento

Tabla 3.1: Páginas principales del frontend React y roles autorizados (elaboración propia).

Componente	Ruta	Roles permitidos
Login.jsx	/	Público
Dashboard.jsx	/dashboard	Todos autenticados
AdminPanel.jsx	/admin	admin
GestorDocumentos.jsx	/gestor	Todos autenticados
Migrantes.jsx	/migrantes	Todos autenticados
CrearMigrante.jsx	/migrantes/crear	admin, coordinador, operador
SolicitudesPush.jsx	/solicitudes	Todos autenticados

3.3.2. Módulo Gestor de Documentos

El componente `GestorDocumentos.jsx` implementa el flujo VOCA completo mediante llamadas `fetch` asíncronas a los endpoints del backend. La Figura 3.3 muestra el estado inicial del módulo con estadísticas por etapa; la Figura 3.4 muestra el formulario de registro de un nuevo documento.

Gestor de documentos

Flujo del proceso: V → O → C → A

Juan Lopezadmin

0Total documentos

0Ventanilla

0Verificación

0Firma

0Finalizados

Nuevo documento

Sube un archivo para iniciar el flujo en etapa Ventanilla.

Título

Descripción

Archivo

Choose File No file chosen

Subir documento

Documentos registrados

Consulta el estado, firma y avance de cada documento.

Título	Etapas	Coordinador	Archivo	Ticket de firma	Acción
No hay documentos disponibles					

Figura 3.3: Vista inicial del Gestor de Documentos con estadísticas por etapa (elaboración propia).

3.3.3. Flujo de firma digital en el frontend

La Figura 3.5 muestra la interfaz de firma digital disponible para el coordinador, y la Figura 3.6 muestra el ticket de firma generado tras la operación criptográfica.

Gestor de documentos

Flujo del proceso: V → O → C → A

Maria Garciaconsulta

0Total documentos
0Ventanilla
0Verificación
0Firma
0Finalizados

Nuevo documento

Sube un archivo para iniciar el flujo en etapa Ventanilla.

Título

Ej. Expediente de validación

Descripción

Agrega una descripción breve del documento

Archivo

Choose File

No file chosen

Subir documento

Documentos registrados

Consulta el estado, firma y avance de cada documento.

Título	Etapas	Coordinador	Archivo	Ticket de firma	Acción
No hay documentos disponibles					

Figura 3.4: Formulario de registro de un nuevo documento en etapa Ventanilla (elaboración propia).

Gestor de documentos

Flujo del proceso: V → O → C → A

Carlos Perezcoordinador

1Total documentos
0Ventanilla
1Verificación
0Firma
0Finalizados

Nuevo documento

Sube un archivo para iniciar el flujo en etapa Ventanilla.

Título

Ej. Expediente de validación

Descripción

Agrega una descripción breve del documento

Archivo

Choose File

No file chosen

Subir documento

Documentos registrados

Consulta el estado, firma y avance de cada documento.

Título	Etapas	Coordinador	Archivo	Ticket de firma	Acción
Expediente de validación Documento de ejemplo para el flujo de revision y firma.	O - Operador / Verificación	Carlos Perez	Ver archivo	Sin firma	Firmar documento

Figura 3.5: Interfaz de firma digital disponible para el coordinador en etapa Operador (elaboración propia).

Gestor de documentos

Flujo del proceso: V → O → C → A

Carlos Perezcoordinador

1Total documentos
0Ventanilla
0Verificación
1Firma
0Finalizados

Nuevo documento

Sube un archivo para iniciar el flujo en etapa Ventanilla.

Título

Ej. Expediente de validación

Descripción

Agrega una descripción breve del documento

Archivo

Choose File

No file chosen

Subir documento

Documentos registrados

Consulta el estado, firma y avance de cada documento.

Título	Etapas	Coordinador	Archivo	Ticket de firma	Acción
Expediente de validacion Documento de ejemplo para el flujo de revision y firma.	C - Coordinador / Firma	Carlos Perez	Ver archivo	FIR-20260604-120000-DOC101-USR2 Firmado por: Carlos Perez Acción: El coordinador firmo digitalmente el documento y lo avanza de Operador a Coordinador Fecha: 2026-06-04 12:00:00	<button>Verificar firma</button>

Figura 3.6: Ticket y folio de firma digital generados tras la operación criptográfica (elaboración propia).

3.4. Conexión de los Componentes

La comunicación entre el frontend React y el backend PHP se realiza mediante el proxy de desarrollo de Vite, configurado en `vite.config.js`:

Listing 3.3: Configuración del proxy de desarrollo en `vite.config.js` (elaboración propia).

```
1 server: {  
2   proxy: {  
3     '/api': {  
4       target: 'http://localhost',  
5       changeOrigin: true,  
6       rewrite: (path) => path  
7     }  
8   }  
9 }
```

Esta configuración permite que el frontend en `localhost:5173` realice llamadas a `/api/*.php` y estas sean redirigidas transparentemente a `http://localhost/casa-monarca-frontend/api/`, eliminando problemas de CORS durante el desarrollo. En producción, el bundle generado por `npm run build` se sirve directamente desde Apache en el mismo dominio que los endpoints PHP.

El flujo completo de una operación representativa —la firma digital de un documento— ilustra la integración de las tres capas:

1. El coordinador presiona **Firmar documento** en `GestorDocumentos.jsx` e ingresa su contraseña de firma.
2. React realiza un `fetch POST` a `/api/firmar_documento.php` con `documento_id`, `usuario_id` y `password_firma` en formato JSON.
3. Vite redirige la petición a Apache, que enruta al archivo PHP correspondiente.

4. PHP recupera la clave privada cifrada de MySQL, la descifra con la contraseña, calcula el hash SHA-256 del archivo físico y firma con `openssl_sign` (RSA-2048).
5. La firma en Base64 y el folio único se almacenan en MySQL; la respuesta JSON incluye `etapa: 5`, el folio y los coordinadores adicionales pendientes.
6. React actualiza el estado del componente y muestra el ticket de firma al coordinador.

Capítulo 4

Resultados

4.1. Resultados del Backend

Los 22 endpoints del backend respondieron correctamente en todos los escenarios de prueba. Los resultados más relevantes son:

- **Autenticación:** `login.php` rechaza correctamente usuarios eliminados, inactivos, revocados y con vigencia vencida, emitiendo mensajes diferenciados para cada caso, sin exponer información de la base de datos.
- **Control de acceso por rol:** la validación de rol en cada endpoint impide que usuarios sin privilegios ejecuten operaciones administrativas, incluso si construyen manualmente la petición HTTP.
- **Firma digital:** `firmar_documento.php` genera firmas RSA-2048 válidas verificables por `verificar_firma_documento.php`. La integridad del archivo se confirma al recalcular SHA-256 en cada verificación, detectando cualquier modificación posterior.
- **Generación de llaves RSA:** `generar_llaves_coordinador.php` genera pares de llaves RSA-2048 con OpenSSL, almacenando la clave pública en PEM y la clave privada cifrada con la contraseña del coordinador.
- **Persistencia:** las operaciones CRUD sobre usuarios, documentos, migrantes y solicitudes se persisten en MySQL con prepared statements, sin vulnerabilidades de inyección SQL.

4.2. Resultados del Frontend

La interfaz React implementa correctamente el flujo completo del sistema:

- El login redirige al dashboard diferenciado por rol; las rutas protegidas bloquean el acceso a páginas no autorizadas.
- El Gestor de Documentos muestra estadísticas por etapa y permite ejecutar todas las acciones del flujo VOCA según el rol del usuario en sesión.
- La firma digital se ejecuta desde el frontend ingresando la contraseña; el ticket resultante se muestra inmediatamente al coordinador.
- La verificación de integridad es accesible para todos los roles y confirma si la firma almacenada corresponde al archivo actual.

La Figura 4.1 muestra la verificación de integridad de un documento firmado; la Figura 4.2 muestra un documento completado en la etapa de Inserción final.

Gestor de documentos

Flujo del proceso: V → O → C → A

1Total documentos

0Ventanilla

0Verificación

1Firma

0Finalizados

Carlos Perezcoordinador

Nuevo documento

Sube un archivo para iniciar el flujo en etapa Ventanilla.

Título

Ej. Expediente de validación

Descripción

Agrega una descripción breve del documento

Archivo

Choose File

No file chosen

Subir documento

Documentos registrados

Consulta el estado, firma y avance de cada documento.

Título	Etapas	Coordinador	Archivo	Ticket de firma	Acción
Expediente de validacion Documento de ejemplo para el flujo de revision y firma.	C - Coordinador / Firma	Carlos Perez	Ver archivo	FIR-20260604-120000-DOC101-USR2 Firmado por: Carlos Perez Acción: El coordinador firmo digitalmente el documento y lo avanza de Operador a Coordinador Fecha: 2026-06-04 12:00:00	Verificar firma

Figura 4.1: Verificación de firma digital: el sistema confirma la integridad del documento (elaboración propia).

4.3. Resultados de la Solución Completa

La solución integra de manera funcional el módulo de identidades (autenticación, RBAC, certificados, vigencias) con el módulo documental (flujo VOCA, firma RSA, verificación SHA-256) y los módulos operativos (registro de migrantes, solicitudes push). El flujo completo de un documento —desde su carga en Ventanilla hasta la Inserción final— opera sin intervención manual fuera del sistema y deja trazabilidad completa en la tabla `documentos_historical`, que registra cada cambio de etapa con el usuario responsable, la fecha y un comentario asociado.

4.4. Casos de Uso y Tabla de Validaciones

Se ejecutaron 15 casos de prueba de caja negra sobre los módulos de identidad y control de acceso. La metodología consistió en definir precondiciones específicas, ejecutar la acción y verificar que la respuesta del sistema coincidiera con el resultado esperado. La Tabla 4.1 resume los resultados con referencia a la evidencia recolectada.

14

<

Tabla 4.1: Matriz de validación y casos de uso del Gestor de Identidades y Documentos (elaboración propia).

15

ID	Descripción	Resultado esperado	Evidencia	Status
7	Usuario eliminado intenta iniciar sesión.	Sistema rechaza con: <i>No existe un usuario con ese correo.</i>	Secc. 3.2.1, Lst. 4.1	Pass
8	Usuario activo y vigente puede iniciar sesión.	Acceso permitido; dashboard visible con rol correcto.	Fig. 3.1, Fig. 3.2	Pass
9	Usuario inactivo intenta iniciar sesión.	Sistema rechaza con: <i>Acceso denegado. El usuario no está activo.</i>	Secc. 3.2.1, Lst. 4.1	Pass
10	Usuario revocado intenta iniciar sesión.	Sistema rechaza con: <i>Acceso denegado. El usuario no está activo.</i>	Secc. 3.2.1, Lst. 4.1	Pass
11	Usuario con vigencia vencida intenta iniciar sesión.	Sistema rechaza con: <i>Acceso denegado. La vigencia ha expirado.</i>	Secc. 3.2.1, Lst. 4.1	Pass
12	Intento de inicio de sesión con contraseña incorrecta.	Sistema rechaza con: <i>Contraseña incorrecta.</i>	Secc. 3.2.1, Lst. 4.1	Pass
13	Correo no registrado intenta iniciar sesión.	Sistema rechaza con: <i>No existe un usuario con ese correo.</i>	Secc. 3.2.1, Lst. 4.1	Pass
14	Persistencia de datos de certificado al crear usuario.	Campos estado, fecha_vigencia y certificado_codigo con valores válidos en MySQL.	Anexo A	Pass
15	Restricción de privilegios para usuario no administrador en operaciones CRUD.	Sistema bloquea crear, editar o eliminar usuarios si el rol no es admin.	Fig. 3.2, Secc. 3.2.1	Pass

Capítulo 5

Conclusiones y Trabajo Futuro

5.1. Conclusiones

La implementación del Gestor de Identidades y Documentos para Casa Monarca demostró que es posible construir un sistema de control de acceso con fundamentos criptográficos sólidos sobre un stack tecnológico accesible (React + PHP + MySQL), sin depender de infraestructura costosa en la nube y con una curva de mantenimiento adecuada para el personal interno de una ONG.

Los 15 casos de prueba de caja negra sobre el módulo de autenticación y control de acceso confirman que el sistema cumple con los objetivos planteados: solo usuarios con estado `activo`, vigencia futura y credenciales correctas pueden autenticarse; las operaciones administrativas están restringidas al rol correspondiente tanto en el frontend (rutas protegidas) como en el backend (validación en cada endpoint); y los certificados digitales únicos generados automáticamente por usuario proveen una base de identidad verificable.

La extensión del alcance hacia la firma digital de documentos con RSA-2048 y SHA-256 aplicó directamente los conceptos estudiados en el bloque: las recomendaciones del NIST SP 800-56B para la fortaleza de pares de llaves, la entropía de Shannon para la aleatoriedad de certificados y el modelo de confianza del SAT para la arquitectura de roles y custodia de llaves privadas. Este módulo responde a una necesidad real del socio formador de contar con un mecanismo objetivamente verificable para la autorización de documentos institucionales.

Desde una perspectiva de seguridad informática, el uso de BCrypt para el hash de contraseñas, de prepared statements para prevenir inyección SQL (OWASP Top 10) y de OpenSSL para la generación y gestión de llaves asimétricas establece una base criptográfica sólida sobre la que pueden construirse versiones futuras del sistema.

5.2. Trabajo Futuro

- **Plan de pruebas extendido:** diseñar casos de prueba formales para los módulos de firma digital, migrantes y solicitudes push, que actualmente operan pero no cuentan con una matriz de validación documentada.
- **Pruebas de aceptación:** realizar pruebas con el socio formador en el entorno de producción (HostGator), verificando en particular la disponibilidad y configuración de la extensión OpenSSL en el servidor de hospedaje.
- **Gestión de sesión:** migrar el almacenamiento de sesión de `localStorage` a tokens JWT con cookies `HttpOnly` para reducir la superficie de ataque ante XSS.

- **Política CORS:** unificar y restringir los headers `Access-Control-Allow-Origin` a dominios específicos en producción, en lugar del comodín * utilizado durante el desarrollo.
- **Panel de auditoría:** aprovechar la tabla `documentos_historial` para construir un módulo de trazabilidad que muestre el historial completo de cambios de etapa de cada documento, con el usuario responsable y la fecha.
- **Notificaciones:** implementar alertas automáticas de vencimiento próximo de certificado o vigencia de usuario, para reducir la carga administrativa del equipo de Casa Monarca.
- **Paginación:** implementar paginación en las tablas de documentos, migrantes y usuarios para garantizar la escalabilidad del sistema conforme crezca el volumen de registros.

Referencias

- [1] Ramió Aguirre, J. (2006). *Libro electrónico de seguridad informática y criptografía* (Versión 4.1). Universidad Politécnica de Madrid.
- [2] Barker, E., Chen, L., Roginsky, A., Vassilev, A., Davis, R., & Simon, S. (2019). *NIST Special Publication 800-56B Revision 2: Recommendation for pair-wise key establishment using integer factorization cryptography*. National Institute of Standards and Technology, U.S. Department of Commerce.
- [3] Shannon, C. E. (2001). A mathematical theory of communication. *ACM SIGMOBILE Mobile Computing and Communications Review*, 5(1), 3–55.
- [4] The PHP Group. (s.f.). *OpenSSL functions*. PHP Manual. <https://www.php.net/manual/es/book.openssl.php>
- [5] The PHP Group. (s.f.). *mysqli prepared statements*. PHP Manual. <https://www.php.net/manual/es/class.mysqli.php>
- [6] React. (2024). *React: The library for web and native user interfaces*. <https://react.dev>
- [7] Vite. (2024). *Vite: Next generation frontend tooling*. <https://vite.dev>
- [8] Casa Monarca, Ayuda Humanitaria al Migrante, A.B.P. (s.f.). *Sitio web oficial*. <https://casamonarca.org.mx/>

Apéndice A

Diccionario de Datos — Tabla usuarios

A continuación se detalla la estructura de la tabla `usuarios`, que es el eje central del módulo de control de acceso basado en roles (RBAC). Esta tabla centraliza la identidad digital de cada colaborador de Casa Monarca, incluyendo los campos criptográficos necesarios para la firma digital de documentos por parte de los coordinadores.

Tabla A.1: Diccionario de datos — tabla `usuarios` (elaboración propia).

Campo	Tipo SQL	Descripción
<code>id</code>	INT AU- TO_INCREMENT	Llave primaria. Identificador único del usuario.
<code>nombre</code>	VARCHAR(255)	Nombre oficial registrado en el sistema.
<code>email</code>	VARCHAR(255) UNIQUE	Correo electrónico; identificador único de inicio de sesión.
<code>password</code>	VARCHAR(255)	Hash BCrypt de la contraseña. Nunca se almacena en texto plano.
<code>rol</code>	VARCHAR(50)	Nivel de privilegios: <code>admin</code> , <code>coordinador</code> , <code>operador</code> o <code>consulta</code> .
<code>estado</code>	VARCHAR(50)	Estado de la cuenta: <code>activo</code> , <code>inactivo</code> o <code>revocado</code> .
<code>fecha_vigencia</code>	DATE	Caducidad temporal de la cuenta. Comparada con la fecha actual en cada autenticación.
<code>certificado_codigo</code>	VARCHAR(100)	Código único generado aleatoriamente al crear el usuario. Identidad digital verificable.
<code>coordinador_id</code>	INT	FK lógica a <code>usuarios.id</code> . Define el coordinador asignado a operadores y consultas.
<code>clave_publica</code>	TEXT	Clave pública RSA del coordinador en formato PEM.
<code>clave_privada_cifrada</code>	TEXT	Clave privada RSA cifrada con la contraseña de firma del coordinador.

Apéndice B

Fragmento de Código — Verificación Criptográfica de Firma

El siguiente fragmento corresponde al endpoint `verificar_firma_documento.php`, que implementa la verificación de integridad de un documento firmado digitalmente. Este proceso recalcula el hash SHA-256 del archivo físico actual y lo valida contra la firma RSA almacenada usando la clave pública del coordinador firmante.

Listing B.1: `verificar_firma_documento.php` — verificación de integridad RSA/SHA-256 (elaboración propia).

```
1 <?php
2 ob_start();
3 header("Access-Control-Allow-Origin: *");
4 header("Content-Type: application/json");
5 require_once '../conexion.php';
6
7 $data = json_decode(file_get_contents("php://input"), true);
8 $documento_id = $data['documento_id'] ?? null;
9
10 // Obtener documento y clave publica del coordinador
11 $stmt = $conexion->prepare(
12     "SELECT d.firma_coordinador, d.hash_archivo,
13        d.archivo_ruta, u.clave_publica
14     FROM documentos d
15     JOIN usuarios u ON d.firmado_por = u.id
16     WHERE d.id = ?"
17 );
18 $stmt->bind_param("i", $documento_id);
19 $stmt->execute();
20 $row = $stmt->get_result()->fetch_assoc();
21
22 // Recalcular hash SHA-256 del archivo fisico actual
23 $hash_actual = hash_file('sha256', $row['archivo_ruta']);
24
25 // Decodificar firma almacenada en Base64
26 $firma_raw = base64_decode($row['firma_coordinador']);
27
28 // Verificar firma RSA con la clave publica del coordinador
29 $clave_publica = openssl_pkey_get_public($row['clave_publica']);
30 $resultado = openssl_verify(
31     $hash_actual,
32     $firma_raw,
33     $clave_publica,
```

```
34     OPENSSL_ALGO_SHA256
35 );
36
37 ob_clean();
38 if ($resultado == 1) {
39     echo json_encode([
40         "success" => true,
41         "valida"   => true,
42         "mensaje"  => "Firma digital valida. El documento no ha sido modificado."
43     ]);
44 } else {
45     echo json_encode([
46         "success" => true,
47         "valida"   => false,
48         "mensaje"  => "Firma invalida o documento modificado."
49     ]);
50 }
51 ?>
```